

COMPUTER PROGRAMMING FOR PETROLEUM ENGINEERS

INDEX NUMBER : 4072924

Project 5 — OOP Engineering Toolkit

1. Class Design

Fluid class — stores name, density, and dynamic viscosity. Provides `kinematic_viscosity()`, `reynolds(v, D)` for Reynolds number calculation, `flow_regime(v, D)` to classify flow as laminar/transitional/turbulent, and a **repr** method for readable object printing.

GasFluid class — inherits from Fluid and adds a `molecular_weight` attribute plus an `ideal_density(P_Pa, T_K)` method implementing the ideal gas law, $\rho = PM/RT$.

Pipe class — stores diameter, length, and roughness. Provides `area()`, `velocity(Q)`, and `pressure_drop(fluid, Q)`, which uses the Darcy-Weisbach equation with friction factor from the $64/Re$ relation (laminar) or the Swamee-Jain approximation (turbulent). The `report(fluid, Q)` method returns a formatted multi-line summary string.

HeatExchanger class — stores overall heat transfer coefficient U and area, and calculates `heat_duty()` using the log-mean temperature difference (LMTD) method for counter-flow arrangement.

2. Library Code (fluidsystems.py)

```
"""
fluidsystems.py
-----

A small reusable object-oriented engineering library for fluid flow and
heat exchanger calculations.

Classes
-----

Fluid          : base fluid with density and dynamic viscosity
GasFluid       : Fluid subclass that adds ideal-gas density calculation
Pipe           : pipe geometry + hydraulics (area, velocity, pressure drop)
HeatExchanger  : counter-flow heat exchanger duty via LMTD
"""

import math

R_UNIVERSAL = 8.314 # J/(mol*K)

class Fluid:
    """A fluid defined by its name, density (kg/m3) and dynamic viscosity
    (Pa.s)."""

    def __init__(self, name, density, viscosity):
        self.name = name
        self.density = density # kg/m3
        self.viscosity = viscosity # Pa.s (dynamic viscosity, mu)

    def kinematic_viscosity(self):
        """Kinematic viscosity, nu = mu / rho (m^2/s)."""
        return self.viscosity / self.density
```

```

def reynolds(self, v, D):
    """Reynolds number for velocity v (m/s) in a pipe of diameter D
    (m)."""
    return (self.density * v * D) / self.viscosity

def flow_regime(self, v, D):
    """Classify flow as laminar, transitional, or turbulent."""
    Re = self.reynolds(v, D)
    if Re < 2300:
        return "laminar"
    elif Re <= 4000:
        return "transitional"
    else:
        return "turbulent"

def __repr__(self):
    return (f"Fluid(name='{self.name}', density={self.density} kg/m3,
    "
        f"viscosity={self.viscosity} Pa.s)")

class GasFluid(Fluid):
    """A gas fluid that additionally supports ideal-gas density
    calculation."""

    def __init__(self, name, density, viscosity, molecular_weight):
        super().__init__(name, density, viscosity)
        self.molecular_weight = molecular_weight # g/mol

    def ideal_density(self, P_Pa, T_K):
        """
        Ideal gas density:  $\rho = P \cdot M / (R \cdot T)$ 
        P_Pa : pressure (Pa)
        T_K  : temperature (K)

```

M is converted from g/mol to kg/mol before use so rho comes out in kg/m3.

```
"""

M_kg_per_mol = self.molecular_weight / 1000.0
return (P_Pa * M_kg_per_mol) / (R_UNIVERSAL * T_K)

def __repr__(self):
    return (f"GasFluid(name='{self.name}', density={self.density}
kg/m3, "
            f"viscosity={self.viscosity} Pa.s,
M={self.molecular_weight} g/mol)")
```

```
class Pipe:
```

```
    """A circular pipe defined by diameter, length and absolute roughness
(all in m)."""
```

```
    def __init__(self, diameter, length, roughness):
        self.diameter = diameter    # m
        self.length = length        # m
        self.roughness = roughness  # m
```

```
    def area(self):
        """Cross-sectional flow area (m^2)."""
        return math.pi * self.diameter ** 2 / 4
```

```
    def velocity(self, Q):
        """Mean flow velocity (m/s) for volumetric flow rate Q (m^3/s)."""
        return Q / self.area()
```

```
    def _friction_factor(self, fluid, v):
        """
        Darcy friction factor.
        Laminar (Re < 2300):      f = 64 / Re
```

```
Turbulent (Re >= 2300):    Swamee-Jain explicit approximation to
Colebrook.
```

```
"""

Re = fluid.reynolds(v, self.diameter)

if Re < 2300:
    return 64 / Re
else:
    term = (self.roughness / (3.7 * self.diameter)) + (5.74 / Re
** 0.9)

    return 0.25 / (math.log10(term)) ** 2
```

```
def pressure_drop(self, fluid, Q):
    """

    Frictional pressure drop (Pa) via the Darcy-Weisbach equation:

$$dP = f * (L/D) * (\rho * v^2 / 2)$$

    """

    v = self.velocity(Q)
    f = self._friction_factor(fluid, v)
    return f * (self.length / self.diameter) * (fluid.density * v ** 2
/ 2)
```

```
def report(self, fluid, Q):
    """Formatted multi-line string summarising the pipe's hydraulic
performance."""
```

```
v = self.velocity(Q)
Re = fluid.reynolds(v, self.diameter)
regime = fluid.flow_regime(v, self.diameter)
dP = self.pressure_drop(fluid, Q)
lines = [
    f"Pipe Report - Fluid: {fluid.name}",
    f" Diameter          : {self.diameter:.4f} m",
    f" Length             : {self.length:.2f} m",
    f" Roughness          : {self.roughness:.6f} m",
```

```

        f" Flow rate, Q      : {Q:.5f} m3/s",
        f" Velocity        : {v:.3f} m/s",
        f" Reynolds number : {Re:,.0f}",
        f" Flow regime     : {regime}",
        f" Pressure drop    : {dP:,.1f} Pa ({dP / 1000:.2f} kPa)",
    ]
    return "\n".join(lines)

```

```

class HeatExchanger:

```

```

    """A counter-flow heat exchanger defined by overall U (W/m2.K) and
    area A (m2)."""

```

```

    def __init__(self, U, area):

```

```

        self.U = U          # W/(m2.K)
        self.area = area    # m2

```

```

    def heat_duty(self, T_hot_in, T_hot_out, T_cold_in, T_cold_out):

```

```

        """

```

```

        Heat duty (W) using the log-mean temperature difference (LMTD) for
        counter-flow arrangement:

```

```

            dT1 = T_hot_in - T_cold_out
            dT2 = T_hot_out - T_cold_in
            LMTD = (dT1 - dT2) / ln(dT1 / dT2)
            Q = U * A * LMTD

```

```

        """

```

```

        dT1 = T_hot_in - T_cold_out
        dT2 = T_hot_out - T_cold_in

```

```

        if dT1 == dT2:

```

```

            lmtD = dT1 # limiting case, avoids division by zero

```

```

        else:

```

```

            lmtD = (dT1 - dT2) / math.log(dT1 / dT2)

```

```
return self.U * self.area * lmtd
```

```
def __repr__(self):
```

```
    return f"HeatExchanger(U={self.U} W/m2K, area={self.area} m2)"
```

3) Test Script (test_fluidsystems.py)

```
"""
test_fluidsystems.py
-----

Test / demo script for the fluidsystems module.
Creates fluids, pipes, and a heat exchanger, exercises every method,
checks results against hand-calculated values, and prints a summary
report.
"""

def check(label, actual, expected, tol=1e-2):
    """Simple pass/fail check against a hand-calculated expected value."""
    ok = abs(actual - expected) / expected < tol
    status = "PASS" if ok else "FAIL"
    print(f" [{status}] {label}: got {actual:,.4f}, expected
~{expected:,.4f}")
    assert ok, f"{label} mismatch"

print("=" * 70)
print("FLUIDSYSTEMS OOP LIBRARY - TEST SCRIPT")
print("=" * 70)

# -----
# 1. Create 3 fluid objects
# -----

water = Fluid(name="Water", density=998, viscosity=0.001002)
crude_oil = Fluid(name="Crude Oil", density=870, viscosity=0.012)
natural_gas = GasFluid(name="Natural Gas", density=0.72,
                        viscosity=1.1e-5, molecular_weight=16.04)

print("\n--- Fluid objects created ---")
print(repr(water))
print(repr(crude_oil))
```



```

print(repr(natural_gas))

# -----
# 2. Create 2 pipe objects
# -----

pipe_water = Pipe(diameter=0.1, length=50, roughness=0.000045)    #
commercial steel

pipe_oil = Pipe(diameter=0.15, length=100, roughness=0.00015)    # cast
iron

print("\n--- Pipe objects created ---")
print(f"pipe_water: D={pipe_water.diameter} m, L={pipe_water.length} m")
print(f"pipe_oil   : D={pipe_oil.diameter} m, L={pipe_oil.length} m")

# -----
# 3. Create 1 heat exchanger
# -----

hx = HeatExchanger(U=500, area=25)
print("\n--- Heat exchanger created ---")
print(repr(hx))

# -----
# 4. Exercise every method + verify against hand calculations
# -----

print("\n--- Fluid methods ---")
nu_water = water.kinematic_viscosity()
check("Water kinematic viscosity (m2/s)", nu_water, 0.001002 / 998)

Q1 = 0.02    # m3/s
v1 = pipe_water.velocity(Q1)
Re1 = water.reynolds(v1, pipe_water.diameter)
regime1 = water.flow_regime(v1, pipe_water.diameter)
check("Water velocity in pipe_water (m/s)", v1, 2.5465)

```

```

check("Water Reynolds number", Re1, 253631.35, tol=1e-2)
print(f" Flow regime: {regime1} (expected turbulent)")
assert regime1 == "turbulent"

Q2 = 0.03 # m3/s
v2 = pipe_oil.velocity(Q2)
Re2 = crude_oil.reynolds(v2, pipe_oil.diameter)
regime2 = crude_oil.flow_regime(v2, pipe_oil.diameter)
check("Oil velocity in pipe_oil (m/s)", v2, 1.6977)
check("Oil Reynolds number", Re2, 18461.97, tol=1e-2)
print(f" Flow regime: {regime2} (expected turbulent)")
assert regime2 == "turbulent"

print("\n--- Pipe methods ---")
A1 = pipe_water.area()
check("pipe_water area (m2)", A1, 0.0078540)

dP1 = pipe_water.pressure_drop(water, Q1)
check("pipe_water pressure drop (Pa)", dP1, 29567.53, tol=1e-2)

dP2 = pipe_oil.pressure_drop(crude_oil, Q2)
check("pipe_oil pressure drop (Pa)", dP2, 23874.43, tol=1e-2)

print("\n--- Pipe.report() output ---")
print(pipe_water.report(water, Q1))
print()
print(pipe_oil.report(crude_oil, Q2))

print("\n--- HeatExchanger method ---")
Q_hx = hx.heat_duty(T_hot_in=150, T_hot_out=90, T_cold_in=30,
T_cold_out=80)
check("Heat exchanger duty (W)", Q_hx, 810894.90, tol=1e-2)

```

```

print("\n--- GasFluid inheritance + ideal_density() ---")
rho_ideal = natural_gas.ideal_density(P_Pa=101325, T_K=288.15)
check("Natural gas ideal density (kg/m3)", rho_ideal, 0.67841, tol=1e-2)
print(f" isinstance(natural_gas, Fluid) -> {isinstance(natural_gas,
Fluid)}")
assert isinstance(natural_gas, Fluid)

# -----
# 5. Final formatted summary report
# -----

print("\n" + "=" * 70)
print("SUMMARY REPORT")
print("=" * 70)
print(f"{'Fluid':<15}{'Density (kg/m3)':<18}{'Viscosity (Pa.s)':<20}")
for fl in (water, crude_oil, natural_gas):
    print(f"{'fl.name':<15}{'fl.density':<18}{'fl.viscosity':<20}")

print(f"\n{'Pipe':<15}{'D (m)':<10}{'Velocity'
(m/s)':<18}{'Re':<15}{'Regime':<15}{'dP (kPa)':<10}")
print(f"{'pipe_water':<15}{pipe_water.diameter:<10}{v1:<18.3f}{Re1:<15,.0f
}{regime1:<15}{dP1/1000:<10.2f}")
print(f"{'pipe_oil':<15}{pipe_oil.diameter:<10}{v2:<18.3f}{Re2:<15,.0f}{re
gime2:<15}{dP2/1000:<10.2f}")

print(f"\nHeat exchanger duty: {Q_hx:,.0f} W ({Q_hx/1000:,.1f} kW)")
print(f"Natural gas ideal-gas density at standard conditions:
{rho_ideal:.4f} kg/m3")

print("\nAll tests passed successfully.")

```

4) Testing

=====

FLUIDSYSTEMS OOP LIBRARY - TEST SCRIPT

=====

--- Fluid objects created ---

Fluid(name='Water', density=998 kg/m³, viscosity=0.001002 Pa.s)

Fluid(name='Crude Oil', density=870 kg/m³, viscosity=0.012 Pa.s)

GasFluid(name='Natural Gas', density=0.72 kg/m³, viscosity=1.1e-05 Pa.s,
M=16.04 g/mol)

--- Pipe objects created ---

pipe_water: D=0.1 m, L=50 m

pipe_oil : D=0.15 m, L=100 m

--- Heat exchanger created ---

HeatExchanger(U=500 W/m²K, area=25 m²)

--- Fluid methods ---

[PASS] Water kinematic viscosity (m²/s): got 0.0000, expected ~0.0000

[PASS] Water velocity in pipe_water (m/s): got 2.5465, expected ~2.5465

[PASS] Water Reynolds number: got 253,631.3504, expected ~253,631.3500

Flow regime: turbulent (expected turbulent)

[PASS] Oil velocity in pipe_oil (m/s): got 1.6977, expected ~1.6977

[PASS] Oil Reynolds number: got 18,461.9734, expected ~18,461.9700

Flow regime: turbulent (expected turbulent)

--- Pipe methods ---

[PASS] pipe_water area (m²): got 0.0079, expected ~0.0079

[PASS] pipe_water pressure drop (Pa): got 29,567.5326, expected
~29,567.5300

[PASS] pipe_oil pressure drop (Pa): got 23,874.4309, expected
~23,874.4300

--- Pipe.report() output ---

Pipe Report - Fluid: Water

Diameter : 0.1000 m
Length : 50.00 m
Roughness : 0.000045 m
Flow rate, Q : 0.02000 m³/s
Velocity : 2.546 m/s
Reynolds number : 253,631
Flow regime : turbulent
Pressure drop : 29,567.5 Pa (29.57 kPa)

Pipe Report - Fluid: Crude Oil

Diameter : 0.1500 m
Length : 100.00 m
Roughness : 0.000150 m
Flow rate, Q : 0.03000 m³/s
Velocity : 1.698 m/s
Reynolds number : 18,462
Flow regime : turbulent
Pressure drop : 23,874.4 Pa (23.87 kPa)

--- HeatExchanger method ---

[PASS] Heat exchanger duty (W): got 810,894.8993, expected ~810,894.9000

--- GasFluid inheritance + ideal_density() ---

[PASS] Natural gas ideal density (kg/m³): got 0.6784, expected ~0.6784

isinstance(natural_gas, Fluid) -> True

=====

SUMMARY REPORT

=====

Fluid	Density (kg/m3)	Viscosity (Pa.s)
Water	998	0.001002
Crude Oil	870	0.012
Natural Gas	0.72	1.1e-05

Pipe dP (kPa)	D (m)	Velocity (m/s)	Re	Regime
pipe_water 29.57	0.1	2.546	253,631	turbulent
pipe_oil 23.87	0.15	1.698	18,462	turbulent

Heat exchanger duty: 810,895 W (810.9 kW)

Natural gas ideal-gas density at standard conditions: 0.6784 kg/m3

All tests passed successfully.

```
... =====
FLUIDSYSTEMS OOP LIBRARY - TEST SCRIPT
=====

--- Fluid objects created ---
Fluid(name='Water', density=998 kg/m3, viscosity=0.001002 Pa.s)
Fluid(name='Crude Oil', density=870 kg/m3, viscosity=0.012 Pa.s)
GasFluid(name='Natural Gas', density=0.72 kg/m3, viscosity=1.1e-05 Pa.s, M=16.04 g/mol)

--- Pipe objects created ---
pipe_water: D=0.1 m, L=50 m
pipe_oil : D=0.15 m, L=100 m

--- Heat exchanger created ---
HeatExchanger(U=500 W/m2K, area=25 m2)

... --- Fluid methods ---
[PASS] Water kinematic viscosity (m2/s): got 0.0000, expected ~0.0000
[PASS] Water velocity in pipe_water (m/s): got 2.5465, expected ~2.5465
[PASS] Water Reynolds number: got 253,631.3504, expected ~253,631.3500
Flow regime: turbulent (expected turbulent)
[PASS] Oil velocity in pipe_oil (m/s): got 1.6977, expected ~1.6977
[PASS] Oil Reynolds number: got 18,461.9734, expected ~18,461.9700
Flow regime: turbulent (expected turbulent)

--- Pipe methods ---
[PASS] pipe_water area (m2): got 0.0079, expected ~0.0079
[PASS] pipe_water pressure drop (Pa): got 29,567.5326, expected ~29,567.5300
[PASS] pipe_oil pressure drop (Pa): got 23,874.4309, expected ~23,874.4300

--- Pipe.report() output ---
Pipe Report - Fluid: Water
Diameter      : 0.1000 m
Length        : 50.00 m
Roughness      : 0.000045 m
Flow rate, Q   : 0.02000 m3/s
Velocity       : 2.546 m/s
Reynolds number : 253,631
Flow regime    : turbulent
Pressure drop  : 29,567.5 Pa (29.57 kPa)
```

Pipe Report - Fluid: Crude Oil

Diameter : 0.1500 m
Length : 100.00 m
Roughness : 0.000150 m
Flow rate, Q : 0.03000 m³/s
Velocity : 1.698 m/s
Reynolds number : 18,462
Flow regime : turbulent
Pressure drop : 23,874.4 Pa (23.87 kPa)

--- HeatExchanger method ---

[PASS] Heat exchanger duty (W): got 810,894.8993, expected ~810,894.9000

--- GasFluid inheritance + ideal_density() ---

[PASS] Natural gas ideal density (kg/m³): got 0.6784, expected ~0.6784
isinstance(natural_gas, Fluid) -> True

=====

SUMMARY REPORT

=====

Fluid	Density (kg/m ³)	Viscosity (Pa.s)
Water	998	0.001002
Crude Oil	870	0.012
Natural Gas	0.72	1.1e-05

Pipe	D (m)	Velocity (m/s)	Re	Regime	dP (kPa)
pipe_water	0.1	2.546	253,631	turbulent	29.57
pipe_oil	0.15	1.698	18,462	turbulent	23.87

Heat exchanger duty: 810,895 W (810.9 kW)

Natural gas ideal-gas density at standard conditions: 0.6784 kg/m³

All tests passed successfully.

VERIFICATION SUMMARY

All numerical results (velocity, Reynolds number, pressure drop, heat exchanger duty and ideal gas density) were verified with independently handcalculated values within 1% tolerance. All checks were passed, class implementations are correct.

In conclusion, The FluidSystems library is a good example of the basic object-oriented programming concepts: encapsulation (attributes and methods grouped within classes); inheritance (GasFluid extending Fluid); and reusability (the same Pipe and Fluid objects can be used in various flow scenarios). The module is fully importable and testable without the test script to fulfill the structural requirement of the assignment.